Cartiae Rae — Full Audit

Date: 2026-08-14 **Scope:** Whole application — payments, auth, data architecture, deployment state, content integrity. **Codebase:** 45 source files, ~14,050 LOC. Live at `cartiaerae.netlify.app`. **Method:** Static review of the repository, plus live probing of the deployed functions and the Supabase schema. Every claim below marked *verified* was tested against production, not inferred from the code.

> **Where this stands.** The audit opened with a critical, exploitable payment flaw on a live Stripe key: the browser sent the price it wanted to pay. That is fixed and verified in production. Order recording — which required Stripe dashboard access the studio does not have — has been rebuilt to work without a webhook. What remains is one SQL migration, one Stripe secret, and two architectural items that deserve scheduling rather than urgency.

Status at a glance

	Finding	State
C1	Browser set the price it paid	**Fixed — verified in production**
H1	Real orders never reached the admin	**Fixed in code** — needs the cleanup migration
H2	Tables used but never created	**Resolved** — migrations written; `site_snapshots` already existed
H3	Production missing required secrets	**1 of 3 left** — `STRIPE_WEBHOOK_SECRET`, now optional
H4	Order recording impossible without Stripe access	**Fixed** — webhook-free path built
H5	Duplicate columns added to `orders`; dollars into cent columns	**Fixed in code** — needs the cleanup migration
M1	Admin roles stored but never enforced	Open — by design for now
M2	`localStorage` is the catalog source of truth	Open — architectural
M3	XSS via admin-controlled content	**Fixed**
M4	Repository is public	Open — currently clean, needs discipline
M5	Expired token embedded in git remote	Open — revoke it
L1	751 KB single JS bundle	Open — cosmetic

CRITICAL

C1 — The browser set the price it paid — FIXED, VERIFIED

Was: `create-checkout-session.js` took `item.price` from the POST body and passed it to Stripe as `unit_amount`. Validation only checked it was a number between 0 and 10000 — never against a catalog. `appliedDiscount.discountPercent` was equally client-supplied and validated nowhere. Anyone could edit the request in devtools and buy a $200 product for Stripe's $0.50 floor.

This was live on a **`pk_live_`** key. Real money.

Now: prices and discount percentages are resolved server-side from the published catalog in `site_snapshots`. The client sends only ids and quantities; name, type and price all come from the server. An unknown id is refused. The function fails **closed** — if the catalog cannot be read it returns 503 rather than falling back to client values.

Verified in production. A tampered cart was posted to the live endpoint:

```
{"id":"ebook-1","name":"HACKED","price":0.5,"quantity":1,
 "appliedDiscount":{"code":"FAKE99","discountPercent":99}}
```

The resulting Stripe page, rendered headlessly:

```
client name "HACKED" honoured : False
real catalog name shown       : True
amounts: ['$24.99']
```

Both the forged price and the fabricated 99% discount were ignored.

HIGH

H4 — Recording orders without Stripe dashboard access — FIXED

The studio has no Stripe dashboard access, so no `STRIPE_WEBHOOK_SECRET` can be obtained and the webhook cannot be registered. Without a webhook, nothing records a sale. Two functions now solve this using only `STRIPE_SECRET_KEY`, which is already configured — creating a webhook needs the dashboard, *reading* a session does not.

`confirm-order` — when the buyer lands on the success page, the server retrieves that session from Stripe and records it only if Stripe itself reports `payment_status: 'paid'`. The browser never asserts a payment. *Verified:* a forged session id returns `"That checkout session could not be found."`

`reconcile-orders` — sweeps recent paid Stripe sessions and backfills any missing, covering buyers who pay and close the tab. Runs automatically when an admin opens the ledger. Admin-only: it verifies the caller's Supabase token maps to an `admin_users` row. *Verified:* both no token and a forged token return `"Administrator access is required."`

All three recorders (`confirm-order`, `reconcile-orders`, `stripe-webhook`) share one code path and upsert on `stripe_checkout_session_id`, so they are safe to run together and duplicates are no-ops.

The honest trade-off: a webhook is server-to-server and always fires. This pair is best-effort plus a sweep, so an order can be minutes late rather than instant. Acceptable for a studio storefront, but the webhook is still worth adding when Stripe access exists — it needs no code change, only the secret.

H5 — Duplicate columns and dollars written into cent columns — FIXED IN CODE

`public.orders` already existed with a complete, well-designed schema: `stripe_checkout_session_id`, separate `payment_status` and `fulfillment_status`, integer-cent amounts, and a companion `order_items` table.

An earlier migration in this repo assumed the table was missing, and when `create table if not exists` did nothing, added parallel columns for facts that already had homes:

Existing	Duplicate that was added
`stripe_checkout_session_id`	`stripe_session_id`
`stripe_payment_intent_id`	`stripe_payment_intent`
`payment_status` + `fulfillment_status`	`status`
`applied_promo_code`	`discount_code`
`applied_discount_percent`	`discount_percent`
the `order_items` table	`items` jsonb

Worse, and much quieter: **`subtotal`, `total`, `discount_total`, `tax_total`, `shipping_total` are `integer` — cents.** The code wrote dollars. Postgres would have silently rounded `24.99` to `25` on every order. No bad data was ever written, because the column mismatch made the writes fail first.

Now: all code writes the original columns, in cents, with line items going to `order_items`. `supabase/orders_schema_cleanup.sql` drops the duplicates — guarded so it raises rather than dropping any column that somehow holds data.

Schema confirmed by probing PostgREST directly, including the types:

```
order_items.unit_price -> invalid input syntax for type integer
orders.total           -> invalid input syntax for type integer
```

H1 — Real orders never reached the admin — FIXED IN CODE

The recorders write verified orders to `public.orders`, but no frontend code read that table — the Orders Ledger rendered React state seeded from `localStorage`. A real customer could pay, the order be recorded correctly, and the studio see an empty ledger forever.

`AppContext` now fetches orders with their `order_items` embedded, converts cents to decimals for display, and "Mark Dispatched" persists to `fulfillment_status` instead of only mutating local state.

Still required: the RLS policies in the cleanup migration. Note that `order_items` needs its **own** read policy — the embedded select returns an *empty array rather than an error* when a policy is missing, so orders would appear with no line items and no visible cause.

H2 — Tables used but never created — RESOLVED

`site_snapshots`, `videos` and `gallery_items` were used by the app with no migration in the repo. `supabase/site_content_setup.sql` now creates all three idempotently with RLS (public read, admin write), and seeds the catalog.

A correction to this finding: `site_snapshots` **already existed and held data** — proven when server-side pricing successfully loaded the catalog from it in production, and confirmed by its `updated_at` of 2026-07-22. Only the *migration* was missing. Run and confirmed: 4 products, 3 eBooks, 2 services, with the existing data preserved.

H3 — Production configuration — 1 OF 3 REMAINING

Live probe results, verbatim:

Function	Response	State
`create-checkout-session`	`"One of the items in your cart is no longer available."`	Working
`confirm-order`	`"That checkout session could not be found."`	Working
`get-ebook-download`	`404 "This order is not confirmed yet."`	Working
`reconcile-orders`	`"Administrator access is required."`	Working
`stripe-webhook`	`500 missing: ["STRIPE_WEBHOOK_SECRET"]`	Blocked, now optional

The Supabase key was set all along under the name `SUPABASE_SECRET_KEY` while the functions read `SUPABASE_SERVICE_ROLE_KEY` — a silent name mismatch that produced a misleading "not configured" error. The functions now accept either name, and `SUPABASE_URL` no longer needs setting at all (it is not a secret; it is already compiled into the public JS bundle).

With H4 delivered, `STRIPE_WEBHOOK_SECRET` is **no longer blocking** — it is an upgrade, not a prerequisite.

MEDIUM

M1 — Roles are stored but never enforced

`admin_users.role` accepts `super_admin` / `store_manager` / `content_manager`, and `auth_full_setup.sql` promotes everyone to `super_admin`. A search for `role ===`, `hasPermission` or `canManage` across `src/` returns **zero matches** (re-verified for this report). Login checks only that a row exists. Every admin can do everything; the three-tier system is presentational.

Acceptable for a single-owner studio. A real problem the day an assistant is given access.

M2 — `localStorage` is the source of truth for the catalog

Products, eBooks, videos, gallery, blogs and services all initialise from `localStorage` (17 distinct keys) with `initialData.ts` as fallback. Supabase is a *snapshot mirror*, not the primary store.

- Two admins on two machines diverge silently; last publish wins.
- Clearing browser data loses unpublished work.
- `localStorage` caps near ~5 MB. There is already quota-handling code at `AppContext.tsx:574`, which means the ceiling has been hit before — base64 images fill it quickly.

This is the largest remaining architectural debt. It is not urgent, and it is not a quick change.

M3 — XSS via admin-controlled content — FIXED

`ServicesPage.tsx` interpolated `service.name` into raw `innerHTML` on image-load failure — admin-editable content executing as markup for every visitor. Now built with `createElement` + `textContent`.

M4 — The repository is public

`themainkeys/Cartie-Rae` is public (`"private": false`). Anything committed is world-readable and indexed. **Currently clean:** no real key values in tracked files, `.env` is gitignored and untracked, and only variable *names* appear in code and documentation. A `service_role` key committed here would bypass every RLS policy in the database.

M5 — An expired token was embedded in the git remote

`.git/config` carried `https://ghp_…@github.com/...` in plaintext, printed by `git remote -v`. It is dead, but it should be **revoked rather than replaced**, and future credentials kept in a helper (`gh auth login`) rather than the URL.

LOW

L1 — Single 751 KB JS bundle

`dist/assets/index-*.js` is ~751 KB (~210 KB gzipped), over Vite's warning threshold. `AdminPortal` is already lazy-loaded (157 KB split out), which was the right instinct; the storefront chunk is the remaining weight.

L2 — Demo session handling is sound

`readValidDemoSession` validates shape and expiry, and demo mode is hard-disabled whenever Supabase is configured. Honestly labelled, correctly gated. No action.

Platform note: Netlify Functions have no WebSocket

Adding `@supabase/supabase-js` to a Netlify Function crashes it at `createClient()`:

```
Node.js 20 detected without native WebSocket support
```

The full client always constructs a Realtime client, which needs a WebSocket that Node 20 lacks. **This briefly took live checkout down during the audit.** The obvious fix does not work: the Functions runtime version is set by `AWS_LAMBDA_JS_RUNTIME`, **not** by `NODE_VERSION` in `netlify.toml`, so changing the build image achieves nothing.

All functions now use PostgREST and Storage directly over `fetch` (`netlify/functions/lib/supabaseRest.js`) — no WebSocket, no realtime, no dependency, smaller bundles. Recorded here because anyone reintroducing `supabase-js` into a function will hit it again.

Content integrity — fabricated data removed

Three sources of invented data were displaying as if real, and are now suppressed whenever the app is connected to a live backend (they remain in demo mode so previews still look alive):

What	Where	Why it mattered
Two customer orders (Aria Carter, Shayla Jenkins)	hardcoded in `AppContext.tsx`	Fake revenue in the studio's own ledger
Like counts + 3 comments on **all 17 videos**	`VideoGallery.tsx`	Customer-facing manufactured social proof
Five-star testimonials on products and eBooks	`initialData.ts`	Invented endorsements attributed to named people

---

Remaining work

#	Item	Blocked on	Priority
1	Run `supabase/orders_schema_cleanup.sql`	Dashboard access	**High** — orders and line items stay invisible without its RLS policies
2	Upload the three eBook PDFs to the private `ebooks` bucket	The files	High — buyers cannot receive what they paid for
3	Place a real test purchase end to end	The above	High — nothing has been proven with a genuine paid order yet
4	Register the Stripe webhook	Stripe access	Medium — an upgrade, no longer a blocker
5	M1 (role enforcement)	Decision	Low until a second admin exists
6	M2 (localStorage architecture)	Scheduling	Low, but growing

Everything else in this report is implemented, deployed and verified.

Assessment

The store can now take money safely — that was not true when this audit began, and it was the single most important thing to change. Order recording is built and deployed but has not yet handled a genuine paid order end to end; that is the next milestone and it depends on item 1 above, not on further development.

The two open architectural items (unenforced roles, `localStorage` as source of truth) are real but not dangerous at the current scale of one owner running one studio. They should be scheduled deliberately rather than rushed.